

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

```

File Edit Format Run Options Window Help
heap_file = []
def insert_record():
    record = input("Enter record to insert: ")
    heap_file.append(record)
    print("Record inserted successfully.")
def delete_record():
    record = input("Enter record to delete: ")
    if record in heap_file:
        heap_file.remove(record)
        print("Record deleted successfully.")
    else:
        print("Record not found.")
def sequential_scan():
    if len(heap_file) == 0:
        print("Heap file is empty.")
    else:
        print("\nRecords in Heap File:")
        for i, record in enumerate(heap_file, start=1):
            print(i, ":", record)
while True:
    print("\n--- Heap File Organization ---")
    print("1. Insert Record")
    print("2. Delete Record")
    print("3. Sequential Scan")
    print("4. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        insert_record()
    elif choice == "2":
        delete_record()
    elif choice == "3":
        sequential_scan()
    elif choice == "4":
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")

```

```

File Edit Shell Debug Options Window Help
>>>
Enter "help" below or click "Help" above for more information.
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/q1_co4.py =====
--- Heap File Organization ---
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter record to insert: student1
Record inserted successfully.
--- Heap File Organization ---
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter record to insert: student2
Record inserted successfully.
--- Heap File Organization ---
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 2
Enter record to delete: student2
Record deleted successfully.
--- Heap File Organization ---
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 3
Records in Heap File:
1 : student1
--- Heap File Organization ---
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 4
Program terminated.
>>>

```

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

```

TABLE_SIZE = 500
hash_table = [[] for _ in range(TABLE_SIZE)]
def hash_function(student_id):
    return student_id % TABLE_SIZE
def insert_student(student_id, name):
    index = hash_function(student_id)
    hash_table[index].append((student_id, name))
    print("Student record inserted successfully.")
def search_student(student_id):
    index = hash_function(student_id)
    for record in hash_table[index]:
        if record[0] == student_id:
            print("Student ID:", record[0])
            print("Student Name:", record[1])
            return
    print("Student record not found.")
def display_records():
    print("\n--- Student Records ---")
    found = False
    for index in range(TABLE_SIZE):
        if hash_table[index]:
            found = True
            print("Index", index, ":", hash_table[index])
    if not found:
        print("No records found.")
while True:
    print("\n--- Static Hashing Student Database ---")
    print("1. Insert Student")
    print("2. Search Student")
    print("3. Display Records")
    print("4. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        student_id = int(input("Enter Student ID: "))
        name = input("Enter Student Name: ")
        insert_student(student_id, name)
    elif choice == "2":
        student_id = int(input("Enter Student ID to search: "))
        search_student(student_id)
    elif choice == "3":
        display_records()
    elif choice == "4":
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")

```

```

--- Static Hashing Student Database ---
1. Insert Student
2. Search Student
3. Display Records
4. Exit
Enter your choice: 1
Enter Student ID: 500
Enter Student Name: shyam
Student record inserted successfully.
--- Static Hashing Student Database ---
1. Insert Student
2. Search Student
3. Display Records
4. Exit
Enter your choice: 1
Enter Student ID: 100
Enter Student Name: aeern
Student record inserted successfully.
--- Static Hashing Student Database ---
1. Insert Student
2. Search Student
3. Display Records
4. Exit
Enter your choice: 2
Enter Student ID to search: 100
Student ID: 100
Student Name: aeern
--- Static Hashing Student Database ---
1. Insert Student
2. Search Student
3. Display Records
4. Exit
Enter your choice: 3
Student Records ---
Index 0 : [(500, 'shyam')]
Index 100 : [(100, 'aeern')]
--- Static Hashing Student Database ---
1. Insert Student
2. Search Student
3. Display Records
4. Exit
Enter your choice: 4
Program terminated.
>>>

```

3.Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

```
records = []
def insert_record():
    student_id = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")
    records.append((student_id, name))
    records.sort(key=lambda x: x[0])
    print("Record inserted successfully.")
def binary_search():
    key = int(input("Enter Student ID to search: "))
    low = 0
    high = len(records) - 1
    while low <= high:
        mid = (low + high) // 2
        if records[mid][0] == key:
            print("\nRecord found:")
            print("Student ID:", records[mid][0])
            print("Student Name:", records[mid][1])
            return
        elif records[mid][0] < key:
            low = mid + 1
        else:
            high = mid - 1
    print("Record not found.")
def display_records():
    if len(records) == 0:
        print("No records available.")
    else:
        print("\n--- Sorted File Records ---")
        for record in records:
            print("Student ID:", record[0], "| Name:", record[1])
while True:
    print("\n--- Sorted File Organization ---")
    print("1. Insert Record")
    print("2. Search Record (Binary Search)")
    print("3. Display Records")
    print("4. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        insert_record()
    elif choice == "2":
        binary_search()
    elif choice == "3":
        display_records()
    elif choice == "4":
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```

```
--- Sorted File Organization ---
1. Insert Record
2. Search Record (Binary Search)
3. Display Records
4. Exit
Enter your choice: 1
Enter Student ID: 123
Enter Student Name: hari
Record inserted successfully.

--- Sorted File Organization ---
1. Insert Record
2. Search Record (Binary Search)
3. Display Records
4. Exit
Enter your choice: 1
Enter Student ID: 235
Enter Student Name: seru
Record inserted successfully.

--- Sorted File Organization ---
1. Insert Record
2. Search Record (Binary Search)
3. Display Records
4. Exit
Enter your choice: 2
Enter Student ID to search: 235

Record found:
Student ID: 235
Student Name: seru

--- Sorted File Organization ---
1. Insert Record
2. Search Record (Binary Search)
3. Display Records
4. Exit
Enter your choice: 3

--- Sorted File Records ---
Student ID: 123 | Name: hari
Student ID: 235 | Name: seru

--- Sorted File Organization ---
1. Insert Record
2. Search Record (Binary Search)
3. Display Records
4. Exit
Enter your choice: 4
```

4.Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

```
class BTreeNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []
        self.leaf = leaf
class BTree:
    def __init__(self, order=4):
        self.root = BTreeNode()
        self.max_keys = order - 1
    def search(self, node, key):
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and key == node.keys[i]:
            return True
        if node.leaf:
            return False
        return self.search(node.children[i], key)
    def split_child(self, parent, index):
        child = parent.children[index]
        new_node = BTreeNode(child.leaf)
        mid = len(child.keys) // 2
        middle_key = child.keys[mid]
        new_node.keys = child.keys[:mid]
        child.keys = child.keys[mid:]
        if not child.leaf:
            new_node.children = child.children[:mid + 1]
            child.children = child.children[mid + 1:]
        parent.keys.insert(index, middle_key)
        parent.children.insert(index + 1, new_node)
    def insert_non_full(self, node, key):
        i = len(node.keys) - 1
        if node.leaf:
            node.keys.append(key)
            node.keys.sort()
        else:
            while i >= 0 and key < node.keys[i]:
                i -= 1
            i += 1
            if len(node.children[i].keys) == self.max_keys:
                self.split_child(node, i)
                if key > node.keys[i]:
                    i += 1
            self.insert_non_full(node.children[i], key)
    def insert(self, key):
        root = self.root
        if len(root.keys) == self.max_keys:
            new_root = BTreeNode(False)
            new_root.children.append(root)
```

```
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/ql_co4.py =====
Inserting keys: [10, 20, 5, 6, 12, 30, 7, 17]

B-Tree Structure:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Enter key to search: 17
Key 17 found in B-Tree.
>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/ql_co4.py =====
Inserting keys: [10, 20, 5, 6, 12, 30, 7, 17]

B-Tree Structure:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Enter key to search: 25
Key 25 not found.
>>>
```

5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

```
class Node:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None
class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = Node(leaf=True)

    def insert(self, key):
        if len(self.root.keys) >= self.order - 1:
            new_root = Node(leaf=False)
            new_root.children.append(self.root)

            self.split_child(new_root, 0)
            self.root = new_root

        self.insert_non_full(self.root, key)

    def insert_non_full(self, node, key):
        if node.leaf:
            node.keys.append(key)
            node.keys.sort()
        else:
            i = len(node.keys) - 1
            while i >= 0 and key < node.keys[i]:
                i -= 1
            i += 1
            if len(node.children[i].keys) >= self.order - 1:
                self.split_child(node, i)

                if key >= node.keys[i]:
                    i += 1

            self.insert_non_full(node.children[i], key)

    def split_child(self, parent, index):
        node = parent.children[index]
        new_node = Node(leaf=node.leaf)

        mid = len(node.keys) // 2

        if node.leaf:
            new_node.keys = node.keys[mid:]
            node.keys = node.keys[:mid]
            new_node.next = node.next
            node.next = new_node
```

```
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/q1_co4.py =====
Leaf Level:
[5, 6, 7] -> [10] -> [12, 17] -> [20] -> [25, 30, 35] -> None
Search 17: True
Search 40: False
Range Query (6 to 25):
[6, 7, 10, 12, 17, 20, 25]
```

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

```
sorted_file = [
    {"id": 101, "name": "Arun"},
    {"id": 102, "name": "Shyam"},
    {"id": 103, "name": "Raj"},
    {"id": 104, "name": "Kumar"},
    {"id": 105, "name": "Vijay"}
]
index_table = {}
for position, record in enumerate(sorted_file):
    index_table[record["id"]] = position
def display_index():
    print("Primary Key\tRecord Position")

    for key, position in index_table.items():
        print(key, "\t\t", position)
def search_by_key(key):
    if key in index_table:
        position = index_table[key]
        record = sorted_file[position]

        print("\nRecord Found:")
        print("ID:", record["id"])
        print("Name:", record["name"])
    else:
        print("\nRecord not found.")

print("DENSE PRIMARY INDEX")
print("-----")

display_index()

search_key = 103

print("\nSearching for key:", search_key)
search_by_key(search_key)
```

```
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/q1_co4.py =====
DENSE PRIMARY INDEX
-----
Primary Key      Record Position
101              0
102              1
103              2
104              3
105              4

Searching for key: 103

Record Found:
ID: 103
Name: Raj
```

7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

```
class Bucket:
    def __init__(self, depth):
        self.local_depth = depth
        self.records = []

class ExtendibleHashing:
    def __init__(self, bucket_size=2):
        self.global_depth = 1
        self.bucket_size = bucket_size
        bucket0 = Bucket(1)
        bucket1 = Bucket(1)
        self.directory = [bucket0, bucket1]
        self.global_depth += 1
    def hash_function(self, key):
        return key & ((1 << self.global_depth) - 1)
    def double_directory(self):
        print("\nDirectory Overflow! Doubling Directory...")
        self.directory = self.directory + self.directory
        self.global_depth += 1
        print("New Global Depth:", self.global_depth)
    def split_bucket(self, index):
        old_bucket = self.directory[index]
        old_depth = old_bucket.local_depth
        if old_depth == self.global_depth:
            self.double_directory()
        new_bucket = Bucket(old_depth + 1)
        old_bucket.local_depth += 1
        for i in range(len(self.directory)):
            if self.directory[i] is old_bucket:
                if (i >> old_depth) & 1:
                    self.directory[i] = new_bucket
        old_records = old_bucket.records
        old_bucket.records = []
        for key in old_records:
            self.insert(key)
    def insert(self, key):
        index = self.hash_function(key)
        bucket = self.directory[index]
        if len(bucket.records) < self.bucket_size:
            bucket.records.append(key)
            return
        self.split_bucket(index)
        self.insert(key)
    def display(self):
        print("\n----- Extendible Hash Table -----")
        print("Global Depth:", self.global_depth)
        print("Directory:")

Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/q1_co4.py =====
Inserting records: [2, 4, 6, 8, 10, 12, 14, 16]

Inserting: 2
----- Extendible Hash Table -----
Global Depth: 1
Directory:
0 -> [2] | Local Depth: 1
1 -> [] | Local Depth: 1

Inserting: 4
----- Extendible Hash Table -----
Global Depth: 1
Directory:
0 -> [2, 4] | Local Depth: 1
1 -> [] | Local Depth: 1

Inserting: 6
Directory Overflow! Doubling Directory...
New Global Depth: 2
----- Extendible Hash Table -----
Global Depth: 2
Directory:
00 -> [4] | Local Depth: 2
01 -> [2] | Local Depth: 1
10 -> [2, 6] | Local Depth: 2
11 -> [] | Local Depth: 1

Inserting: 8
----- Extendible Hash Table -----
Global Depth: 2
Directory:
00 -> [4, 8] | Local Depth: 2
01 -> [2] | Local Depth: 1
10 -> [2, 6] | Local Depth: 2
11 -> [6] | Local Depth: 1

Inserting: 10
Directory Overflow! Doubling Directory...
```

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

```
import math
total_records = 1000
records_per_block = 10
total_blocks = math.ceil(total_records / records_per_block)
record_number = 750
sequential_io = math.ceil(record_number / records_per_block)
indexed_io = 2
print("SECONDARY STORAGE BLOCK ACCESS SIMULATION")
print("-----")
print("Total Records:", total_records)
print("Records per Block:", records_per_block)
print("Total Blocks:", total_blocks)
print("Record to Retrieve:", record_number)

print("\nSequential Retrieval")
print("I/O Operations Required:", sequential_io)

print("\nIndexed Retrieval")
print("I/O Operations Required:", indexed_io)

print("\nComparison")
print("Sequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", indexed_io)

Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/q1_co4.py =====
SECONDARY STORAGE BLOCK ACCESS SIMULATION
-----
Total Records: 1000
Records per Block: 10
Total Blocks: 100
Record to Retrieve: 750

Sequential Retrieval
I/O Operations Required: 75

Indexed Retrieval
I/O Operations Required: 2

Comparison
Sequential Retrieval I/O: 75
Indexed Retrieval I/O: 2
>>>
```

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

```
import time
import random
sorted_file = []

for i in range(1, 10001):
    sorted_file.append((i, "Student_" + str(i)))
dense_index = {}

for position, record in enumerate(sorted_file):
    key = record[0]
    dense_index[key] = position

block_size = 100
sparse_index = {}

for position in range(0, len(sorted_file), block_size):
    key = sorted_file[position][0]
    sparse_index[key] = position

def dense_search(key):
    if key in dense_index:
        position = dense_index[key]
        return sorted_file[position]
    return None

def sparse_search(key):
    start_position = 0

    for index_key in sparse_index:
        if index_key <= key:
            start_position = sparse_index[index_key]
        else:
            break

    end_position = min(start_position + block_size, len(sorted_file))

    for position in range(start_position, end_position):
        if sorted_file[position][0] == key:
            return sorted_file[position]

    return None

search_key = 7856
start_time = time.perf_counter()
result_dense = dense_search(search_key)

Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/ql_co4.py =====
SPARSE INDEX VS DENSE INDEX
-----

Search Key: 7856

Dense Index Result: (7856, 'Student_7856')
Dense Index Search Time: 5.400041118264198e-06 seconds

Sparse Index Result: (7856, 'Student_7856')
Sparse Index Search Time: 1.810002140700817e-05 seconds

Dense Index Entries: 10000
Sparse Index Entries: 100

>>>
```

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

```
class BPlusNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None

class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(leaf=True)

    def insert(self, key):
        if len(self.root.keys) == self.order - 1:
            new_root = BPlusNode(leaf=False)
            new_root.children.append(self.root)
            self.split_child(new_root, 0)
            self.root = new_root

        self.insert_non_full(self.root, key)

    def insert_non_full(self, node, key):
        if node.leaf:
            node.keys.append(key)
            node.keys.sort()
        else:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1

            if len(node.children[i].keys) == self.order - 1:
                self.split_child(node, i)

            if key >= node.keys[i]:
                i += 1

            self.insert_non_full(node.children[i], key)

    def split_child(self, parent, index):
        node = parent.children[index]
        new_node = BPlusNode(leaf=node.leaf)
        mid = len(node.keys) // 2

Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/sr051/OneDrive/Desktop/CSA0506-DBMS/ql_co4.py =====
B+ Tree Traversal:
5 10 15 20 25 30 35 40 45 50 55
Range Query (15 to 45):
[15, 20, 25, 30, 35, 40, 45]

>>>
```